* Stack Based Expression Evaluation :→ (Postfix)

Let's try to understand Postfix Expression :→

Normal Mathematics :→ (infix) (BODMAS)

$2+3*4 = 2+12 = 14$

Postfix Form :→ 234*+

Rules of postfix :→

* Numbers come first
* Operators come after operands

Example :→ Expression :→ 23*54*+9−

Meaning : $(2*3)+(5*4)-9$
$= 6+20-9$
$= 17$

Algorithm (Stack ⇒ LIFO)

Scan left → right

1. If number → push to stack
2. If operator → pop 2 numbers ③ Perform Operation
4. Push the result back to stack

DRY RUN : expression → [23*54*+9−] (string) → 17 (int)

Step	Symbol	Stack	Operation
1	2	2	push
2	3	2 3	push
3	*	6	2×3 = 6
4	5	6 5	push
5	4	6 5 4	push
6	*	6 20	4×5 = 20
7	+	26	6+20 = 26
8	9	26 9	push
9	−	17	26 − 9 = 17

'0' → 48
'1' → 49
49 − 48 = 1
A A ↓

Final Answer = 17

Queue Data Structure :→ (First In First Out)
(Last In Last Out)

* Use case : [Data Sequencing]

Queue using Array :→ int MAX-SIZE = 10;

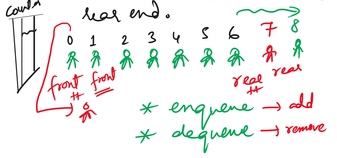
Two variables:

front f
rear r

Single element

* The front element normally goes out.
* We add elements at the rear end.

count



isEmpty ( ){
return (front == −1 && rear == −1);
}

isFull ( ){
return (rear == MS−1);
}

* enqueue → add
* dequeue → remove

front −1, rear, empty ①	Single element ②	③
	x, f 0 rear, 0 − 0 + 1 = 1	x y z a b, 0 1 2 3 4, 5 elements, front, rear, 4 − 0 + 1 = 5

① Case 1 : Empty :→ return → 0 size = 0

r − f + 1 case 1
(−1) − (−1) + 1
−1 + 1 + 1
= 1

return → size = rear − front + 1;

Sliding Window Problem :→ windows (n − k + 1) = 8 − 3 + 1 = 5 + 1 = 6

* Find the maximum element of each sliding window in the given array if the window size is k = 3 : {TCS / Capgemini / Bosch / Vonbar / HCL} (2024 − 2025) (2026)

arr = { 1, 3, −1, −3, 5, 3, 6, 7 }; k = 3 { window size }

		Mx	
①	[1 3 −1] −3 5 3 6 7	3	0 to n − k
②	1 [3 −1 −3] 5 3 6 7	3	0 to 8 − 3
③	1 3 [−1 −3 5] 3 6 7	5	0 to 5
④	1 3 −1 [−3 5 3] 6 7	5	(0, 1, 2, 3, 4, 5)
⑤	1 3 −1 −3 [5 3 6] 7	6	6 windows
⑥	1 3 −1 −3 5 [3 6 7]	7	

o/p [3 3 5 5 6 7]

{ Draw a DRY RUN Table for the Sliding Window Max }

{ 1, 3, −1, −3, 5, 3, 6, 7 } k = 3